LAB: Stepper Motor

LAB: Stepper Motor

Date: 2026-10-12

Author/Partner: Choi Jung Hyun

Demo Video: https://youtu.be/14XiaOTzq5k

PDF version:

Introduction

In this lab, we will learn how to drive a stepper motor using the digital GPIO outputs of an MCU. You will use a Finite State Machine (FSM) to design the algorithm for stepper motor control.

You must submit

- LAB Report (*.pdf & *.md)
- Zip source files(main*.c, ecRCC.h, ecGPIO.h, ecSysTick.c etc...).
    - Only the source files. Do not submit project files

Requirement

Hardware

- MCU
    - NUCLEO-F411RE
- Actuator/Sensor/Others:
    - **Tutorial:**
        - Stepper Motor SM-42BYG011 (12V)
        - Motor Driver A4988
    - **Lab:**
        - Stepper Motor 28BYJ-48 (5V)
        - Motor Driver ULN2003
    - breadboard

Software

- Keil uVision, CMSIS, EC_HAL library

Tutorial: Stepper Motor Driver (A4988)

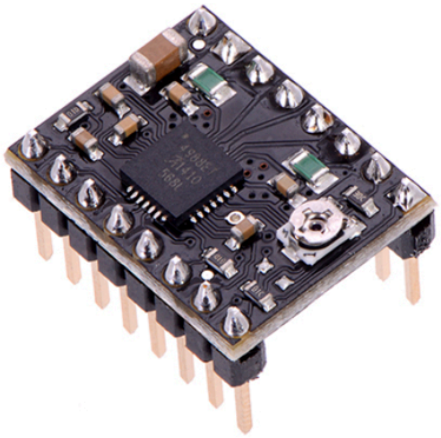
A simple method to drive a stepper motor is using a motor driver that receives only a pulse and direction pin.

- **A4988 motor driver** A4988 spec sheet
- **DRV8825**

- Any motor shield based on A4988, DRV8825

This can also change the micro-stepping configuration easily. For more information, read

{% embed url="https://www.makerguides.com/a4988-stepper-motor-driver-arduino-tutorial/" %}



A4988 / DRV8825



Driver Shield
based on A4988 / DR8825

The stepper motor for the tutorial can be any unipolar or bipolar (<24V)

- Stepping Motor (Bipolar 12V, SM-42BYG011-25)
- Stepping Motor (5V, KH4248 b90112: spec sheet) etc

Connection Diagram

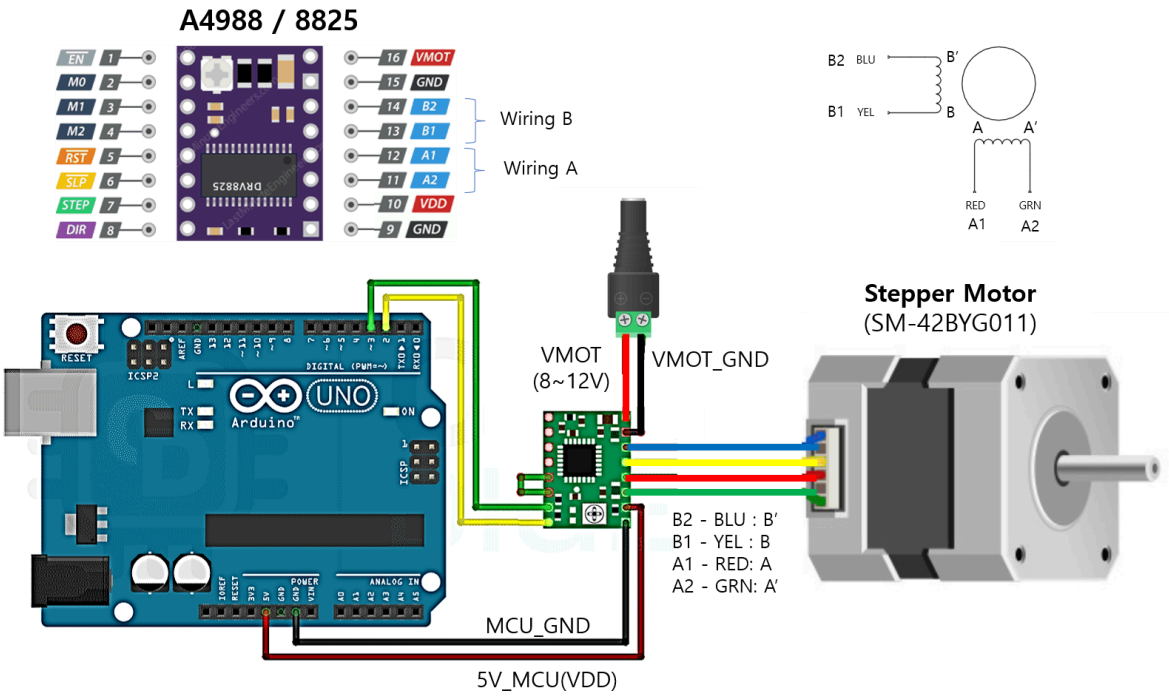
(1) Connection Diagram (A4988)

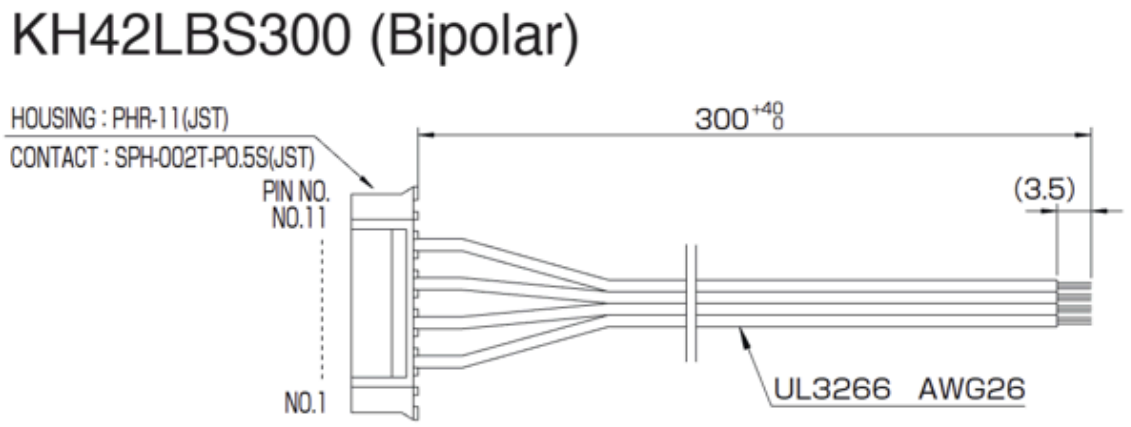
1. Connect the motor driver and the stepper motor as follows.
2. Supply 8V from a power supply to VMOT(#16 on Driver)

Functions	Motor	Driver	MCU-Arduino	MCU-STMf411
Step		#7	D2	PA_10
Direction		#8	D3	PB_3
VDD (MCU 5V)		#9	5V	+5V
VDD_GND (MCU GND)		#10	GND	GND
Vmotor (PowerSupply)		#16 to Power(8V), Capacitor 100uF (pin#15-16)		
Vmotor_GND (PowerSupplyGnd)		#15 to Power(GND)		

Functions	Motor	Driver	MCU-Arduino	MCU-STMf411
Motor A, A'	A: RED, A':GRN	#12 to A, #11 to A'		
Motor B, B'	B: YEL, B':BLU	#13 to B, #14 to B'		
Others		#5(RST) to #6(SLP)		

> DO NOT connect pin#15 to pin#9 (MCU_GND)!!
>
> Motor wire Color code can be different. Just find the pair A-A' and B-B'
>
> B1-B2: the samewiring, A1-A2: the same wiring
>
> It does not matter to invert the motor wiring A-A’ (pin A1-A2) to A’-A (pin A1-A2)





PHR-11 (Pin No.)	3	5	7	9
Excitation (PHASE)	A	$\overline{A}$	B	$\overline{B}$
Cable Color	Red	Blue	Yellow	White

(2) Connection Diagram (Motor Driver Shield based on A4988)

Pin Connection

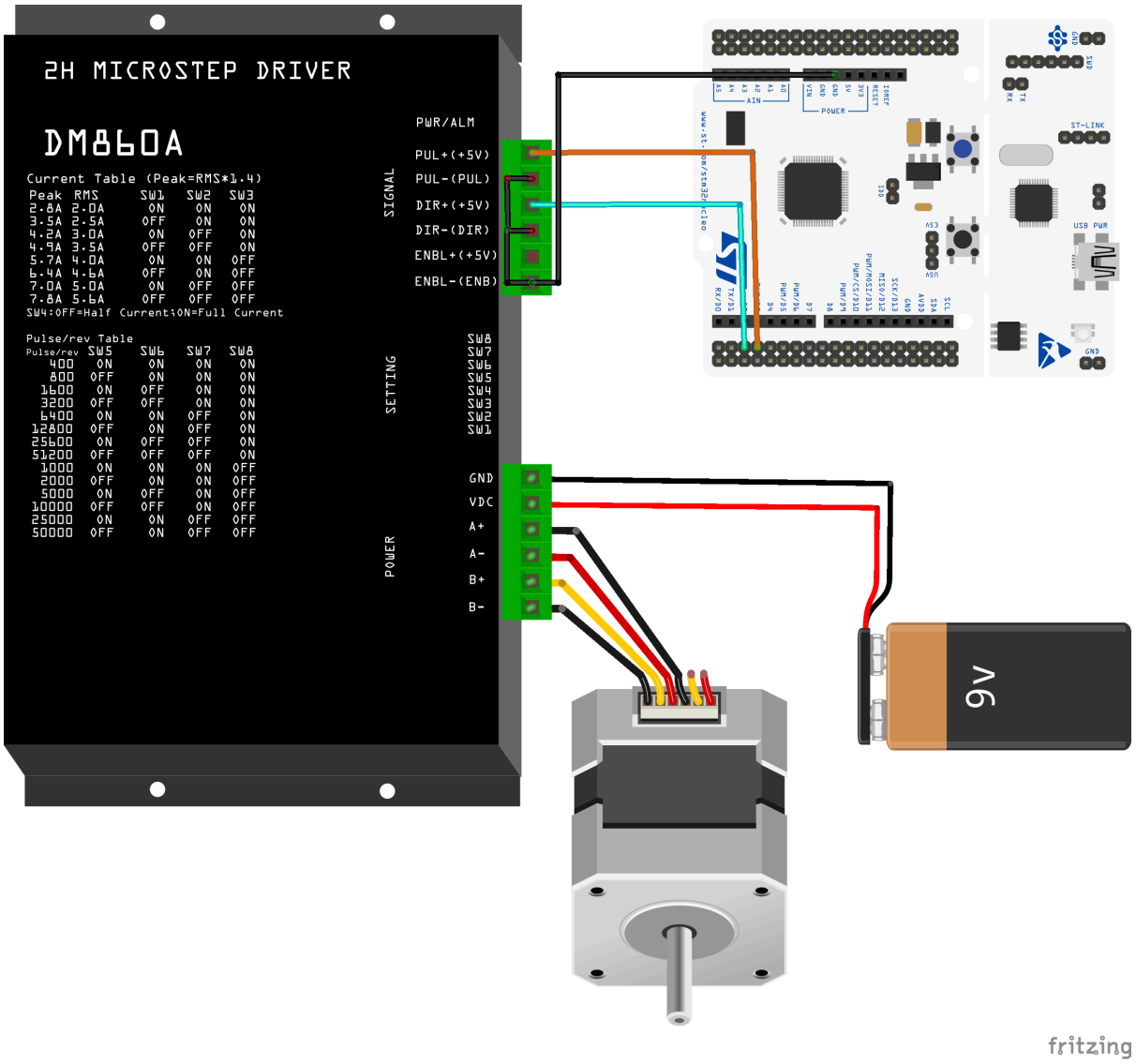
MCU:

- DIR: D2(PA10), STEP: D3(PB3), ENABLE: D6(PB10)

Power Supply: 8~12V

Depends on the motor shield

- SW4 to SW6 : ON (Lowest Current RMS)
- SW1 to SW3: x1 micro-stepping



Tutorial Program

For Arduino

Create a new project under the directory `tutorial\`

Open *Arduino IDE* and Create a new program named as '**TU_arduino_Stepper.ino**'.

For STM M4

Create a new project under the directory `tutorial\`

Create a new program named as '**TU_Stepper_Motor.c**'.

Run the sample code

{% tabs %} {% tab title="Arduino " %} {% code expandable="true" %}

```

```

{% endcode %}

```
// Run Stepper Motor with A4998

int x;
#define BAUD (9600)
const int dirPin = 2;
const int stepPin = 3;
const int enablePin = 6;


void setup()
{
  Serial.begin(BAUD);
  pinMode(enablePin,OUTPUT); // Enable
  pinMode(stepPin,OUTPUT); // Step
  pinMode(dirPin,OUTPUT); // Dir
  digitalWrite(enablePin,LOW); // Set Enable low
}

void loop()
{
  digitalWrite(dirPin, HIGH);// Set Dir high (CW)
  Serial.println("Loop 200*5 steps (1*5 rev)");
  for(x = 0; x < 1000; x++) // Loop 200 times
  {
    digitalWrite(stepPin,HIGH); // Output high
    delay(1); // Wait
    digitalWrite(stepPin,LOW); // Output low
    delay(1); // Wait
  }
  Serial.println("Pause");
  delay(1000); // pause one second
}
```

{% endtab %}

{% tab title="STM " %} {% code expandable="true" %}

```
#include "stm32f411xe.h"
#include "ecRCC2.h"
#include "ecGPIO2.h"
#include "ecSysTick2.h"

#define MOTOR_DIR PB_3
#define MOTOR_PULSE PA_10

void setup(void)
{
    RCC_PLL_init();
```

```
    SysTick_init();
    GPIO_init(MOTOR_DIR, OUTPUT);
    GPIO_init(MOTOR_PULSE, OUTPUT);
}


int main(void) {
    setup();
    // Set Dir high (CW)
    GPIO_write(MOTOR_DIR, HIGH);
    while(1){
          for(x = 0; x < 1000; x++)
          {
            GPIO_write(MOTOR_PULSE, HIGH);
            delay_ms(10);
            GPIO_write(MOTOR_PULSE, LOW);
            delay_ms(10);
          }
        delay_ms(1000);
    }
}
```

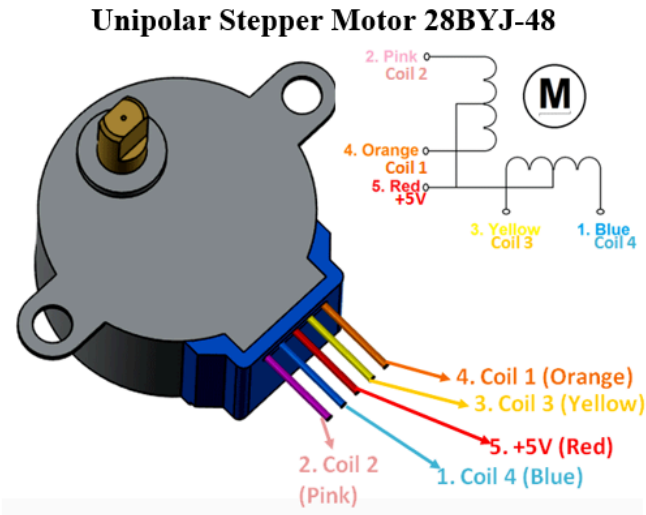
{% endcode %} {% endtab %} {% endtabs %}

Problem : Stepper Motor Control with FSM

Problem Description

For the lab, we are going to use another type of stepper motor driver that requires a sequence of 4-input pulses as the input.

- **Driver: ULN2003 motor driver.**ULN2003 spec sheet
- **Motor:** 28BYJ-48, Search for the spec sheet



- Rated Voltage: 5V DC
- Number of Phases: 4
- Stride Angle: 5.625°/64
- Gear ratio: 1/32
- Pull in torque: 300 gf.cm
- Coil: Unipolar 5 lead coil

1. Find out the number of steps required to rotate 1 revolution of the stepper motor with Full-steppping.

2. Then, rotate the stepper motor 5 revolutions with 1 rpm.
3. Repeat the above process in the opposite direction.
4. Increase and decrease the speed of the motor as fast as it can rotate to find the maximum and minimum speed of the motor.
5. Apply the half-stepping and repeat the above.

FSM design for Stepper Motor Sequence

We are going to apply the finite state machine method to drive a stepper motor.

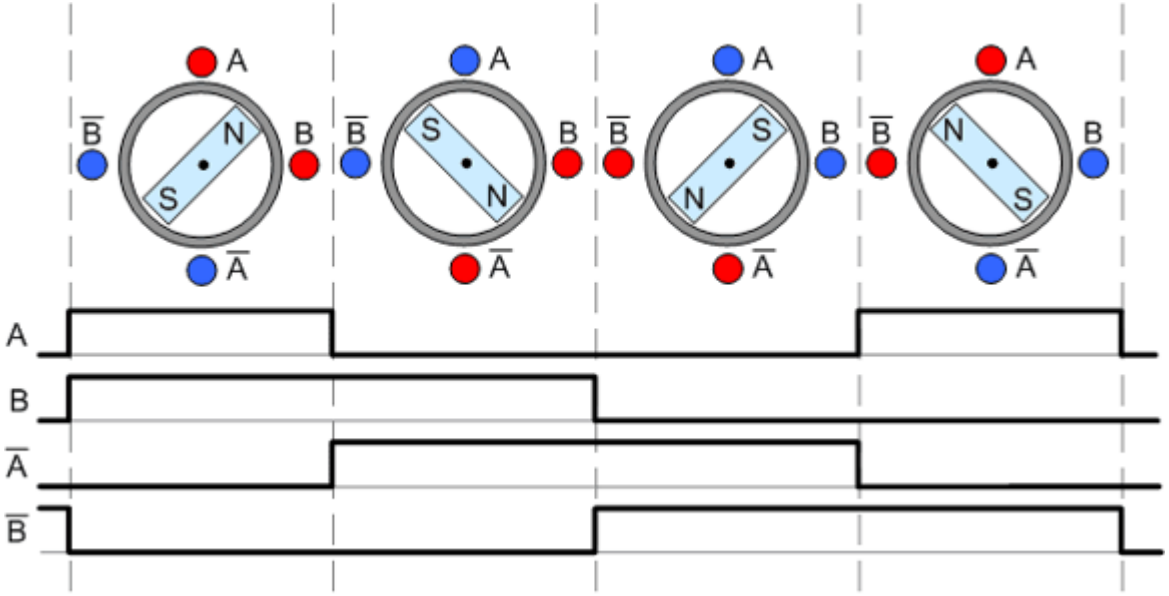
You must read Tutorial: FSM programming for hints

{% content-ref url="/pages/kqR8sgzWXvRPlhf4Vkz6" %} Tutorial: Finite State Machine programming {% endcontent-ref %}

State Table for Full-Stepping

Fill-in the blanks of the given tables

Full-stepping sequence



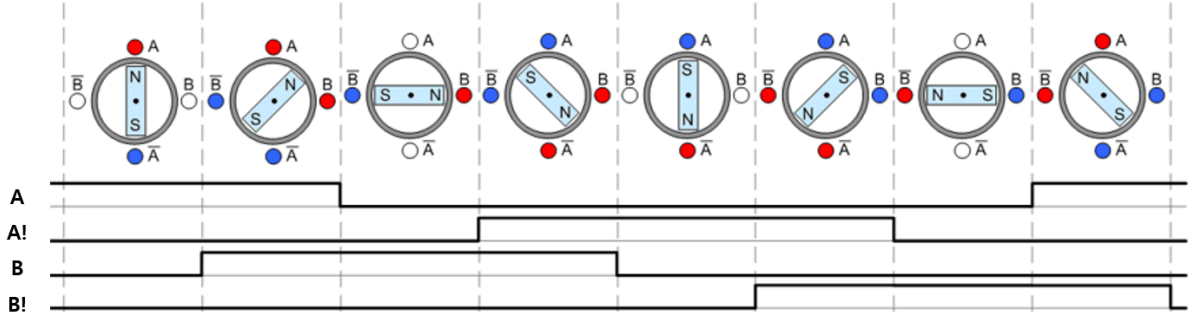
Phase	Port_Pin	Sequence			
		1	2	3	4
A	PB_10	H	L	L	H
B	PB_4	H	H	L	L
A’	PB_5	L	H	H	L
B’	PB_3	L	L	H	H

State Table: Moore FSM

State	Next State		Output
	DIR=0	DIR=1	(A B A′ B′)
S0	S1	S3	1100
S1	S2	S2	0110
S2	S3	S1	0011
S3	S0	S0	1001

State Table for Half-Stepping

Half-stepping sequence



Phase	Port_Pin	Sequence							
		1	2	3	4	5	6	7	8
A	PB_10	H	H	L	L	L	L	L	H
A′	PB_4	L	L	L	H	H	H	L	L
B	PB_5	L	H	H	H	L	L	L	L
B′	PB_3	L	L	L	L	L	H	H	H

- **State Table**: Moore FSM

State	Next State		Output
	DIR=0	DIR=1	(A A′ B B′)
S0	S1	S7	1000
S1	S2	S6	1010
S2	S3	S5	0010
S3	S4	S4	0110
S4	S5	S3	1011
S5	S6	S2	0101
S6	S7	S1	0001
S7	S0	S0	1001

Programming FSM

Preparation

- Download files:
  - ecStepper_student.h, ecStepper_student.c
- Change the library files as `ecStepper.h, ecStepper.c`
- Write your name and modified date in the comment section
- must update your header files located in the directory `include\`.
- Create a new project under the directory `EC\lab\`
  - Environment: "**LAB_Stepper_Motor**"
  - Source File: "**LAB_Stepper_Motor.c**"
- You must modify the `platformio.ini` **,** to add new environment.

> You MUST write your name in the top of the source file, inside the comment section.

Function Definitions

Fill-in the blanks in the header files to complete the definitions for the provided functions

{% tabs %} {% tab title="ecStepper.h " %}

```
//State number
typedef enum {
    S0, S1, S2, S3, S4, S5, S6, S7
} StateNum;


// Stepping sequence  - FSM
typedef struct {
  uint32_t next[2];
  uint8_t out[4];
} StateStepper_t;

// Initialization of Stepper Motor
void stepper_init(uint32_t mode, PinName_t *pinStepper);

// Converts Motor [rpm] to step delay in [msec]
void stepper_speed (uint32_t speedRPM);

// Run Stepper Motor for given steps and direction
void stepper_step(uint32_t steps, uint32_t direction);

// Stepper Motor Output for given state
void stepper_out(uint32_t state);

// Stop Stepper Motor
void stepper_stop(void);
```

{% endtab %}

{% tab title="ecStepper.c " %} {% code expandable="true" %}

```
// Output: A B A' B'
static const StateStepper_t fsm_Full[4] = {
 {{S1,S3},{HIGH,HIGH,LOW,LOW}},
    // YOUR CODE
    // YOUR CODE
    // YOUR CODE
};


static const StateStepper_t fsm_Half[8] = {
    {{S1,S7},{1,0,0,0}},
    // YOUR CODE
    // YOUR CODE
    // YOUR CODE
    // YOUR CODE
};


// Initialization of Stepper Motor
void stepper_init(uint32_t mode, PinName_t *pinStepper){
    if (mode == HALF)
        fsm = fsm_Half;
    else
        fsm = fsm_Full;

    //  GPIO Digital Out Initiation
    // For A, B, AN, BN

    for (int i=0; i<4; i++){
        stepperPins[i] = pinStepper[i];
        // YOUR CODE
        // YOUR CODE
    }
}

// Converts Motor [rpm] to step delay in [msec]
void stepper_speed (uint32_t speedRPM){      // rpm
        _step_delay = 60*1000/(STEP_PER_REV*speedRPM);      //in [msec]
        // [usec] can be used for a higher accuracy
        // But needs delay_us()
}


// Run Stepper Motor for given steps and direction
void stepper_step(uint32_t steps, uint32_t direction){
    // Exit if stepper_init() not called
    if (fsm == 0) return;
    _steps = steps;
```

```
    // run for step size:
    for (uint32_t i = 0; i < _steps; i++){
        // Update Present State
        // YOUR CODE

        // Output of Present State
        stepper_out(state);

        // Delay for steppermotor speed
        delay_ms(_step_delay);
    }
}

// Stepper Motor Output for given state
void stepper_out(uint32_t state){

    // stepperPins[0]=A, stepperPins[1]=B, ...
    // out[0]=A, out[1]=B, ...

    // YOUR CODE
    // YOUR CODE
}


// Stop Stepper Motor
void stepper_stop(void) {
    _steps = 0;

    // All pins DigitalOut '0'
    // YOUR CODE
    // YOUR CODE
}
```

{% endcode %} {% endtab %} {% endtabs %}

> Note that these are **blocking** stepper controllers.
>
> While the stepper is running, the MCU cannot process other polling commands. If you can, modify it to be the non-blocking controller.

> You can also create your own functions different from the given instructions.

Code

Your code goes here: [ADD Code LINK such as github] (https://github.com/ChoiJunghyun0818/EC-STM32F4-22100750/blob/main/lab/LAB_Stepper_Motor) (https://github.com/ChoiJunghyun0818/EC-STM32F4-22100750/blob/main/include/ecStepper.h) (https://github.com/ChoiJunghyun0818/EC-STM32F4-22100750/blob/main/include/ecStepper.c)

Explain your source code with necessary comments.

```
/*----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : [Choi Jung Hyun]
Created          : 05-03-2021
Modified         : 10-12-2026
Language/ver     : C++ in VS Code

Description      : Main code for controlling stepper motor
/----------------------------------------------------------------*/


#include "stm32f411xe.h"
#include "ecGPIO2.h"
#include "ecRCC2.h"
#include "ecEXTI2.h"
#include "ecSysTick2.h"
#include "ecStepper.h"

// ( A, B,  AN,  BN)
PinName_t pinStepper[4]    = {PB_10,PB_4,PB_5,PB_3};

volatile uint32_t motorRunning = 1;

// Initialiization
void setup(void){
    RCC_PLL_init();                                 // System Clock = 84MHz
    SysTick_init();                                 // Systick init

    GPIO_init(PC_13, INPUT);                // GPIOC pin13 initialization
    EXTI_init(PC_13, FALL,0);                   // External Interrupt Setting

    stepper_init(HALF, pinStepper);                 // Stepper GPIO pin
initialization
    // this part is changed based on the full or half -> problem 6
    stepper_speed(15);                                  //  set stepper motor
speed

    GPIO_init(PC_13, INPUT);  // Replace BT2_PIN with your actual pin
    EXTI_init(PC_13, FALL, 0);

}
// problem 2. stepper speed 1 rotates the stepper motor about 1 revolutions with 1
rpm.
// meaning 1 full rotation takes a minute
// problem 4. the max speed is 14
// problem 6. the max speed is 29 for half


int main(void) {
    // Initialiization --------------------------------------------------------
    setup();
```

```
    uint32_t rev=1;
    uint32_t steps=STEP_PER_REV*2; // this 2 is aslo changed when half(problem 6)

    // Inifinite Loop ----------------------------------------------------------
    while(1){
        stepper_step(2048, DIR_CW);  // (Step : 2048, Direction : 0 or 1, Mode :
FULL or HALF)
        delay_ms(2000);
        stepper_step(2048, DIR_CCW);  // (Step : 2048, Direction : 0 or 1, Mode :
FULL or HALF)
        delay_ms(2000);
    }
}

// problem 1. 2048 steps are needed for the 1 rotation full stepper
// problem 3. doing problem 2 the opposite way around is poosible with DIR_CCW
with a 2 second delay



void EXTI15_10_IRQHandler(void)
{
    if (is_pending_EXTI(PC_13)) {
        if (motorRunning) {
            motorRunning = 0;
            stepper_emergency_stop();
        } else {
            motorRunning = 1;
            stepper_resume();
        }

        clear_pending_EXTI(PC_13);
    }
}
```

Sample Code : Stepper Motor

{% code expandable="true" %}

```
#include "stm32f411xe.h"
#include "ecGPIO2.h"
#include "ecRCC2.h"
#include "ecEXTI2.h"
#include "ecSysTick2.h"
#include "ecStepper2.h"

// ( A, B,  AN,  BN)
PinName_t pinStepper[4]    = {PB_10,PB_4,PB_5,PB_3};

// Initialiization
void setup(void){
```

```
    RCC_PLL_init();                                 // System Clock = 84MHz
    SysTick_init();                                 // Systick init

    GPIO_init(BUTTON_PIN, EC_DIN);                  // GPIOC pin13 initialization
    EXTI_init(BUTTON_PIN, FALL,0);                  // External Interrupt Setting

    stepper_init(FULL, pinStepper);                 // Stepper GPIO pin
initialization
    stepper_speed(2);                               //  set stepper motor speed
}


int main(void) {
    // Initialiization --------------------------------------------------------
    setup();
    uint32_t rev=1;
    uint32_t steps=STEP_PER_REV*rev;

    // Inifinite Loop ---------------------------------------------------------
    while(1){
        stepper_step(steps, DIR_CW);  // (Step : 2048, Direction : 0 or 1, Mode :
FULL or HALF)
        delay_ms(2000);
        stepper_step(steps, DIR_CCW);  // (Step : 2048, Direction : 0 or 1, Mode :
FULL or HALF)
        delay_ms(2000);
    }
}



void EXTI15_10_IRQHandler(void) {
    if (is_pending_EXTI(BUTTON_PIN)) {
        stepper_stop();
        clear_pending_EXTI(BUTTON_PIN); // cleared by writing '1'
    }
}
```

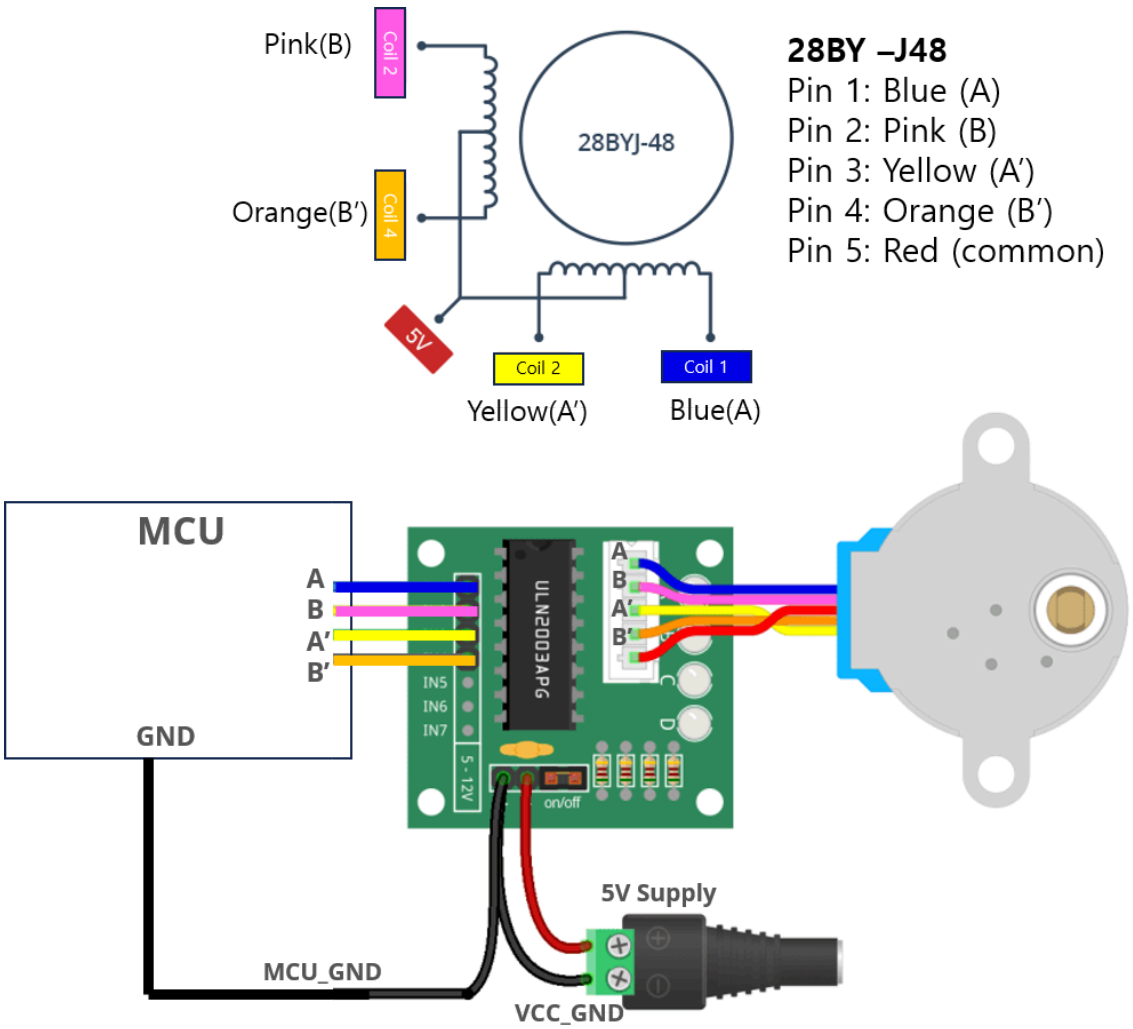
{% endcode %}

Configuration

Function	Pins	Configuration
A, B, A', B'	PB10, PB4, PB5, PB3	DOUT, FAST

Connection Diagram

Read the specification sheet of the motor and the motor driver for wiring and min/max input voltage/current.



Discussion

1. Find out the trapezoid-shape velocity profile for a stepper motor. When is this profile necessary?

> A trapezoid-shape velocity profile for a stepper motor is a speed control method where the motor accelerates at a constant rate, operates at a steady target speed, and decelerates at a constant rate back to a stop. This profile is necessary to overcome initial rotor inertia and prevent missed steps or motor stalls during startup, eliminate overshoot from kinetic momentum when stopping, and enable smooth positioning under heavy dynamic loads or high operating speeds.

2. How would you change the code more efficiently for micro-stepping control? You don't have to code this but need to explain your strategy.

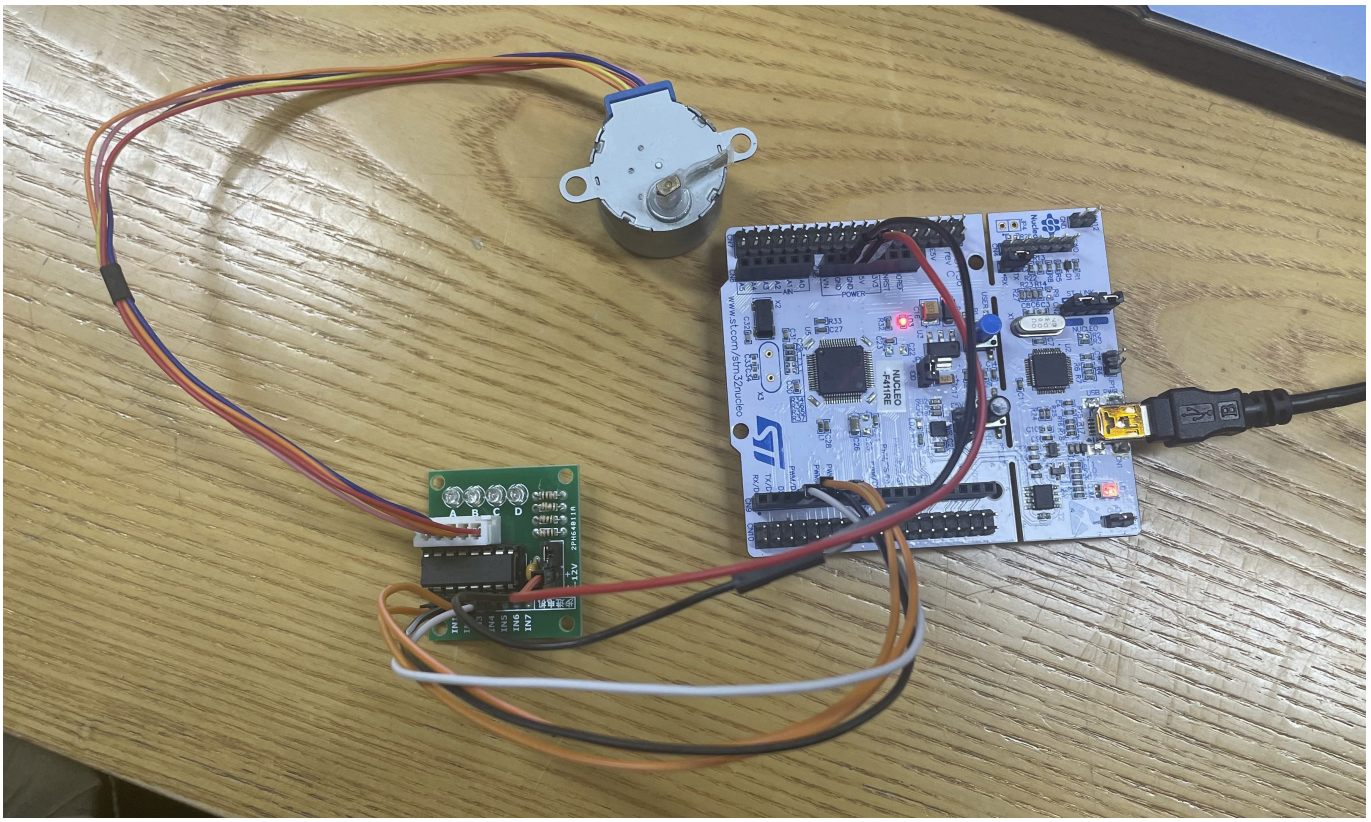
> I would focus on replacing on-the-fly math with pre-computed data and leveraging internal hardware peripherals to handle timing. Instead of calculating trigonometric sine and cosine functions or PWM duty cycles during runtime—which causes severe CPU bottlenecks, storing these values in a pre-computed look-up table (LUT) inside Flash memory so the code only needs to read the next array index in a single clock cycle.

3. There are other types of Stepper Motor Drivers that are simple to use, such as you only give one pulse signal and direction, instead of giving 4 pulse signals. Such examples are A4988, DRV 8834, and TB6600 drivers. Compare these motor drivers with ULN2003 in terms of operating method.

> drivers like the A4988, DRV8834, and TB6600 operate using an efficient STEP/DIR interface where the microcontroller needs to send only two signals—a single digital pulse to execute each step on the STEP pin. Also they use a HIGH/LOW state on the DIR pin to dictate rotational direction—allowing the onboard driver IC to autonomously handle internal motor coil sequencing, current limiting, and hardware microstepping across two GPIO pins. In contrast, the ULN2003 is a passive Darlington transistor array requiring direct phase switching, where the microcontroller must consume four GPIO pins and actively run software to sequence electrical pulses directly to all four motor coils in precise order.

Results

Experiment images and results



Add demo video link

Reference

Complete list of all references used (github, blog, paper, etc)

Troubleshooting

(Option) You can write Troubleshooting section